CSE311L · DATABASE SYSTEMS LAB · NORTH SOUTH UNIVERSITY

SkillSwap NSU
Setup, architecture and workflow

How to run the project on XAMPP, how the front end, the PHP layer and MySQL fit together, and what every screen of the website actually does.

Repository `github.com/mahedi-123/SkillSwapNSU`
Static preview `mahedi-123.github.io/SkillSwapNSU/`

SkillSwap NSU is a student skill-exchange platform. A student lists what they can teach and what they want to learn, the platform finds the pairs where each side has something the other wants, they book a session, and afterwards they rate each other. No money is involved anywhere in the system.

This document has six parts. The first gets the project running on XAMPP from a copied folder. The second explains how a click in the browser turns into a row in MySQL and back again. The third covers the database itself. The fourth walks through every screen and says what it does. The fifth follows one exchange from first request to final rating, naming the SQL at each step. The sixth is a suggested order for demonstrating the project.

Contents

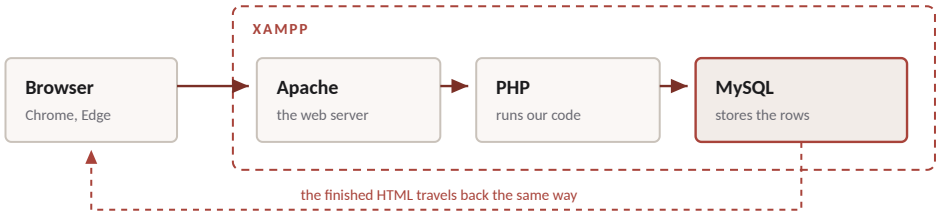
PART I

Running it on XAMPP

1 What XAMPP provides, and why this project needs nothing else

XAMPP is a single installer that bundles three separate programs. **Apache** is the web server: it listens for browser requests and decides what to send back. **PHP** is the language Apache hands a `.php` file to, so that the file can be executed rather than merely sent. **MySQL** (MariaDB, in current XAMPP builds) is the database server that actually stores the rows. **phpMyAdmin** comes along as a web interface for that database.

That is the entire stack this project uses. Nothing has to be downloaded, installed or built beyond XAMPP itself, and the site does not need an internet connection either — Bootstrap, the icon font and the three typefaces are stored inside the project folder under `static/vendor/`. Copying the folder onto a machine that has XAMPP is genuinely the whole deployment.



phpMyAdmin is a separate web page, also served by Apache, that talks to MySQL for you.
It is how the database is imported and inspected — the website itself never goes through it.

Apache receives the request, PHP builds the page, MySQL supplies the data.

2 The four setup steps

1 Copy the folder into htdocs

Apache only serves files that live in one particular directory, and on a standard Windows install that directory is `C:\xampp\htdocs`. Copy the `skillswap` folder from the pen drive into it, so that this file exists:

```
C:\xampp\htdocs\skillswap\index.php
```

The folder name becomes part of the address, so a folder called `skillswap` is reached at `localhost/skillswap/`. Any name works as long as the address matches.

2 Start Apache and MySQL

Open the XAMPP Control Panel and press **Start** next to **Apache** and next to **MySQL**. Both names turn green when they are running. Apache alone is not enough: the site will load but every page will report that the database cannot be reached.

3 Import the database

Open `http://localhost/phpmyadmin` in the browser. Go to the **Import** tab, press **Choose File**, select `database/skillexchange_full.sql` from inside the project folder, and press **Go**.

That one file does everything: it drops any earlier copy of the database, creates the `skillexchange` database, creates all six tables with their primary keys, foreign keys, CHECK and UNIQUE constraints and indexes, creates the three views, and inserts the sample data. Importing `schema.sql` and then `seed.sql` separately produces exactly the same result, and is worth doing once if the schema itself is being discussed.

4 Open the site and sign in

Go to `http://localhost/skillswap/`. Press **Open the demo**, or **Sign in** directly.

Email any address from the `users` table, for example `mahedi.shakib@northsouth.edu`
Password `password123` — all fifty seeded students share it

The **Fill demo credentials** button on the sign-in page enters a working pair. Account 1, Mahedi Hasan, is also the account the admin console runs as, so signing in as that student is the most useful starting point.

> THE ONE FILE YOU MAY NEED TO EDIT
>
> `includes/config.php` holds the database host, user, password and name. The values in it are the XAMPP defaults — user `root` with an empty password — so on a stock XAMPP install nothing needs changing. If a particular machine's MySQL has a root password, put it in `DB_PASS` and nothing else has to move.

3 Checking that the import worked

In phpMyAdmin, click `skillexchange` in the left column. The table list should show six tables with these row counts, and the three views below them.

TABLE	ROWS	WHAT IT HOLDS
`users`	50	One row per student
`skills`	50	The catalogue students choose from
`userskills`	261	Which student teaches or wants to learn which skill
`exchangerequests`	49	One proposed trade
`sessions`	34	A booked meeting belonging to a request
`reviews`	36	Feedback after a completed session

The same numbers appear on the website's admin console, which is the quickest way to confirm from the other side that the two are looking at the same database. If the counts match in both places, the setup is correct.

4 When something does not work

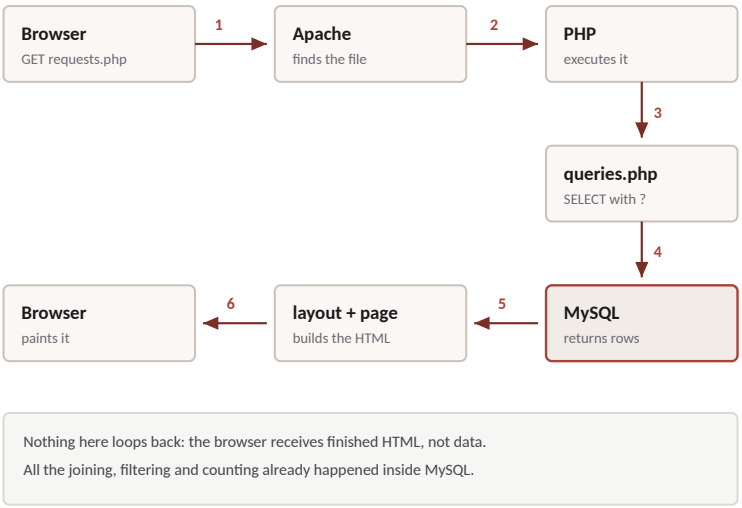
WHAT YOU SEE	WHAT IT MEANS AND WHAT TO DO
A page headed “SkillSwap cannot reach MySQL”	PHP is running, so Apache is fine; the database connection failed. Almost always MySQL is not started, or the SQL file has not been imported yet. The page lists the three causes in order of likelihood.
Apache will not start	Port 80 is already taken, usually by Skype or IIS. The Control Panel's *Netstat* button names the program. Either stop it, or change Apache's port, in which case the address becomes `localhost:8080/skillswap/`.
MySQL will not start	Port 3306 is taken by another MySQL installation, or a previous shutdown left the data directory locked. Stopping the other MySQL service is the usual fix.
A completely blank white page	PHP hit an error and is configured not to display it. Uncomment the last line of `includes/config.php` and reload to see the message.
The page loads but has no styling	The folder was copied incompletely — `static/` is missing or partial. Copy the whole folder again.
“Object not found” on `localhost/skillswap/`	The folder is not inside `htdocs`, or its name does not match the address.

PART II

How the pieces connect

5 What happens between the click and the page

This is the question worth being precise about, because it is where the three technologies meet. Suppose a student clicks **Requests** in the navigation bar. Seven things happen, in order.



One page request, from the click to the painted screen.

One. The browser sends `GET /skillswap/requests.php` to Apache. **Two.** Apache sees the `.php` extension and, instead of sending the file's bytes, hands it to PHP to execute. **Three.** The page calls a function in `includes/queries.php`, which prepares a SELECT statement carrying `?` placeholders and binds this student's id to it. **Four.** mysqli sends that statement to MySQL, which does the joining, filtering, counting and ordering and returns the finished rows. **Five.** The page loops over those rows and writes HTML, wrapped in the shell that `includes/layout.php` provides. **Six.** Apache sends that HTML back. **Seven.** The browser paints it.

> THE POINT WORTH MAKING TO AN EXAMINER
>
> The browser never sees the database, never receives a table, and never does any joining. It receives a finished page. Every list on the site was ordered by MySQL's `ORDER BY`, every count came from `COUNT()`, and every filter was a `WHERE` clause — none of it is JavaScript. Turning JavaScript off entirely leaves the site fully usable, apart from the animations on the landing page.

6 What each file is responsible for

The project is deliberately split so that no single file does two jobs. The eleven screens contain markup and almost no logic; the SQL lives in one file; the writes live in another; the shared chrome lives in a third.

FILE	RESPONSIBILITY
`index.php`	The public landing page. Every number on it is a real query result.
`login.php` · `logout.php`	Sign in, sign out
`register.php`	Create an account, inside a transaction
`dashboard.php`	The signed-in home: matches, requests to answer, next sessions
`search.php`	Find a partner by skill, category, department, level or name
`profile.php`	A student's public profile, addressed as `profile.php?id=19`
`edit-profile.php`	Own details, own skill lists, password change
`requests.php`	Exchange requests received, sent and all
`sessions.php`	Sessions upcoming, completed and cancelled, plus booking
`reviews.php`	Reviews received and written, and sessions still awaiting one
`admin.php`	Admin console: students, skills, requests, review moderation
`actions.php`	**Every write in the whole application arrives here as a POST.**
`includes/config.php`	Database credentials — the only file that ever needs editing
`includes/db.php`	The mysqli connection, and the helpers `rows()`, `row()`, `val()`, `run()`
`includes/auth.php`	Sign in, the session, PBKDF2 hashing, CSRF tokens, flash messages
`includes/queries.php`	**Every SELECT the site runs**, as parameterised SQL
`includes/helpers.php`	Escaping, dates, avatars, status pills, star rows, swap cards
`includes/layout.php`	The page shell: head, top bar, left rail, footer, toasts
`static/css/style.css`	The whole theme
`static/js/app.js`	Presentation only — no data work happens in the browser
`static/vendor/`	Bootstrap, icons and fonts, so the site runs offline
`database/`	The four SQL files

7 The read path, traced through one screen

The requests screen is a good specimen because it is small enough to read in full and still shows every idea. It begins by reading the two filters out of the URL, because both the direction tab and the status dropdown are ordinary links and an ordinary form rather than JavaScript:

```
$me     = me_id();
$dir    = q('dir', 'received');     // received | sent | all
$status = q('status', '');          // Pending, Accepted, ...

$counts = request_status_counts($me);
$list   = requests_of($me, $dir, $status);
```

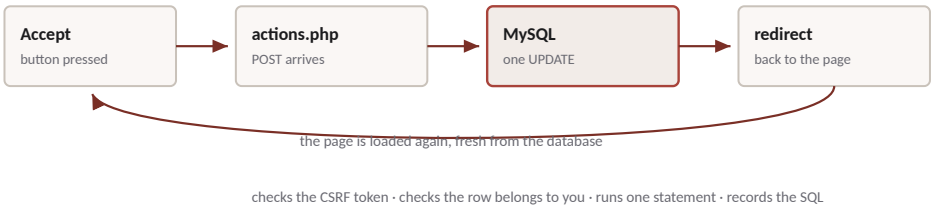
`requests_of()` lives in `includes/queries.php` and assembles one statement. Every value is a placeholder, and the direction and status only decide *which clauses are present*, never what is pasted into them:

```
SELECT     er.*,
           CASE WHEN er.sender_id = ? THEN 'sent' ELSE 'received' END AS dir,
           snd.name AS sender_name,   rcv.name AS receiver_name,
           ts.skill_name AS teach_name, ls.skill_name AS learn_name,
           (SELECT COUNT(*) FROM sessions se
             WHERE se.request_id = er.request_id
               AND se.status <> 'Cancelled')            AS booked
FROM       exchangerequests er
INNER JOIN users  snd ON snd.user_id = er.sender_id
INNER JOIN users  rcv ON rcv.user_id = er.receiver_id
INNER JOIN skills ts  ON ts.skill_id = er.teach_skill
INNER JOIN skills ls  ON ls.skill_id = er.learn_skill
WHERE      er.receiver_id = ?          -- or sender_id, or both
  AND      er.status = ?               -- only when a status was chosen
ORDER BY   er.created_at DESC
```

That single statement joins five tables and carries a correlated subquery, so one round trip to MySQL produces everything the screen needs: the request, both students' names, both skill names, and how many sessions have already been booked against it. The page then loops over the returned array and writes cards. There is no second query inside the loop, which is the mistake this arrangement is designed to avoid.

8 The write path, and why it redirects

Reads are spread across the eleven pages; writes are not. Every button that changes data — accepting a request, booking a session, publishing a review, adding a skill, deleting a student — posts to the same file, `actions.php`, with a hidden field naming the action.



POST, redirect, GET — so refreshing the page never repeats the action.

Inside `actions.php` each action does the same four things in the same order. It verifies the CSRF token, so a form from another site cannot trigger it. It verifies ownership — you cannot accept a request addressed to somebody else, however the id is edited. It runs exactly one parameterised statement, or one transaction where two statements must travel together. And it records a message naming the SQL that ran, before redirecting the reader back to the page they came from.

```
case 'request.status': {
    $id     = $int('request_id');
    $status = $str('status');

    $r = request_by_id($id);
    if (!$r || ($r['sender_id'] != $me && $r['receiver_id'] != $me)) {
        flash('That request is not yours to change.', '', 'bad');
        bounce($back);
    }

    run('UPDATE exchangerequests SET status = ? WHERE request_id = ?', [$status, $id]);
    flash("Request #$id is now $status.",
          "UPDATE exchangerequests SET status = '$status' WHERE request_id = $id");
    bounce($back);
}
```

The redirect at the end is not decoration. Without it the browser's address bar would still hold a POST, and pressing refresh would ask “resend the form?” and run the statement a second time. Because the action always ends in a redirect to an ordinary GET, refreshing simply reloads the page. This pattern is called POST, redirect, GET.

> THE MESSAGE THAT NAMES THE STATEMENT
>
> Every action leaves a small message in the corner of the next page giving the SQL it just ran — accepting request 22 reports `UPDATE exchangerequests SET status = 'Accepted' WHERE request_id = 22`. That is not a mock-up; it is the statement that executed a moment earlier. It means the data layer can be demonstrated without opening the source code, and it is the single most useful thing to point at during a viva.

When two statements must travel together

Completing a session is the one place where a single click has to change two tables. If it was the last open session of an exchange, the exchange itself is finished too. Those two updates are wrapped in a transaction, so either both happen or neither does:

```
tx_begin();
try {
    run('UPDATE sessions SET status = ? WHERE session_id = ?', [$status, $id]);

    if ($status === 'Completed') {
        $open = val('SELECT COUNT(*) FROM sessions
                     WHERE request_id = ? AND status = \'Scheduled\'', [$requestId]);
        if ($open === 0) {
            run('UPDATE exchangerequests SET status = \'Completed\'
                 WHERE request_id = ?', [$requestId]);
        }
    }
    tx_commit();
} catch (Throwable $e) {
    tx_undo();
}
```

9 Signing in, and what a session actually is

The `users` table never contains a password. It contains a PBKDF2-SHA256 hash: a one-way fingerprint, produced by running the password through 260,000 rounds of hashing with a random salt. A row looks like this, truncated:

```
pbkdf2:sha256:260000$F5ftweqbJ7GX9leT$e1fec41c78c1de6778c48b9a...
```

Signing in therefore does not compare passwords. It fetches the stored hash for that email address, runs the submitted password through the same 260,000 rounds with the same salt, and compares the two results. Nothing can turn the stored value back into the password — which is the whole point, and is why a leaked database is not a leaked password list.

```
SELECT user_id, name, password FROM users WHERE email = ?
```

```
if (password_check($submitted, $row['password'])) {
    session_regenerate_id(true);
    $_SESSION['user_id'] = (int) $row['user_id'];
}
```

On success PHP stores the student's id in `$_SESSION`, which lives *on the server*. The browser only receives a random session identifier in a cookie — never the id, never the name, never anything that could be edited to become somebody else. Every page then begins with `require_login()`, which redirects to the sign-in page when that session slot is empty. Signing out destroys the session.

Registration is the mirror image. The chosen password is hashed before the `INSERT`, so the plain text never reaches the database, and the whole thing runs as a transaction because creating an account writes to two tables:

```
START TRANSACTION;
INSERT INTO users (name, email, password, department, bio) VALUES (?, ?, ?, ?, ?);
INSERT INTO userskills (user_id, skill_id, skill_type, proficiency) VALUES (?, ?, 'Teach', ?);
INSERT INTO userskills (user_id, skill_id, skill_type, proficiency) VALUES (?, ?, 'Learn',
'Beginner');
COMMIT;
```

10 Three defences built into every request

Prepared statements

Every value in every statement in this project is a `?` placeholder, bound separately by mysqli. Nothing is ever concatenated into SQL. The reason is not tidiness. If a search box built its query by joining strings, a student could type `' OR 1=1 --` into it and change the meaning of the statement. With a placeholder, MySQL is given the query shape first and the values second, so a value can never become syntax — that text is searched for literally and finds nothing. This is SQL injection, and binding is the defence against it.

Output escaping

Anything that came out of the database passes through `e()` before it is printed, which converts `<`, `>`, `&` and quotes into HTML entities. A student whose bio contains a `<script>` tag has it displayed as text rather than executed. That is cross-site scripting, and escaping is the defence against it.

CSRF tokens

Every form that changes data carries a hidden random token that is also held in the server-side session, and `actions.php` refuses any POST whose token does not match. Without it,

another website could quietly submit a form to `localhost/skillswap/actions.php` while the student was signed in.

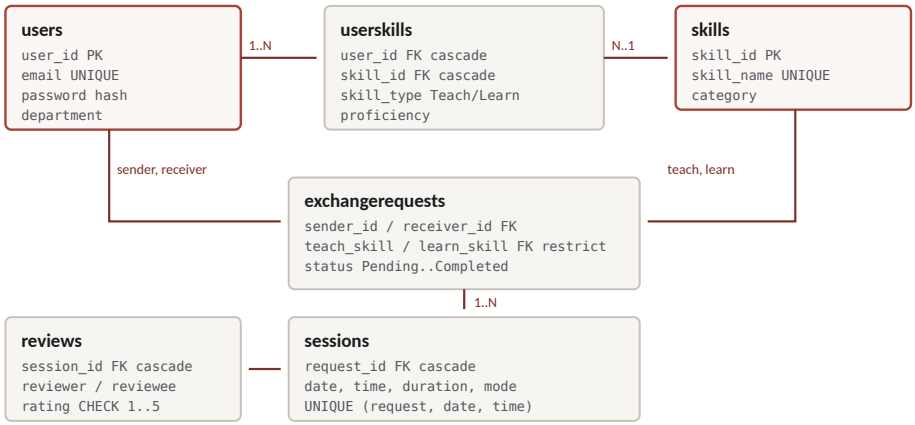
Alongside those three, the application checks ownership on every write, and MySQL enforces its own rules underneath — which is the subject of the next part.

PART III

The database

11 Six tables and how they relate

The design is six tables in third normal form. Students and skills have a many-to-many relationship, resolved through the `userskills` bridge table. An exchange request names two students and two skills. A session belongs to one request. A review belongs to one session.



Six tables. The bridge table is what keeps the many-to-many relationship in 3NF.

The normalisation argument is short. Every column holds a single atomic value, so a student who teaches three skills produces three rows in `userskills` rather than a comma-separated list — that is first normal form. Every table has a single-column surrogate primary key, so no partial dependency can exist — second normal form. And no fact is stored twice: a skill's category lives only in `skills`, and a student's average rating is not stored on `users` at all but derived through a view — third normal form.

12 Constraints that do real work

The most useful thing to demonstrate about this database is that its rules are enforced by MySQL, not by PHP. Each of the following can be triggered from the website in a few clicks, and in each case the application does not prevent the action — it attempts it, MySQL refuses, and the refusal is reported in plain English.

CONSTRAINT	WHAT IT STOPS, AND WHERE TO SEE IT
`UNIQUE (request_id, session_date, session_time)`	Booking the same slot twice for one exchange. Book a session, then book the identical date and time again on the sessions screen.
`UNIQUE (session_id, reviewer_id)`	Reviewing the same session twice. Both partners may review each other, but neither can review twice.
`UNIQUE (user_id, skill_id, skill_type)`	Listing the same skill twice under the same type, on the edit profile screen.
`CHECK (rating BETWEEN 1 AND 5)`	A rating outside the scale.
`CHECK (duration BETWEEN 15 AND 480)`	A session of nine hours, or of two minutes.
`CHECK (reviewer_id <> reviewee_id)`	Reviewing yourself.
`CHECK (sender_id <> receiver_id)`	Sending a request to yourself.
`ON DELETE CASCADE`	Orphan rows. Deleting a student in the admin console runs one `DELETE FROM users`, and their skills, requests, sessions and reviews go with them — with no further SQL from the application.
`ON DELETE RESTRICT`	Removing a skill that an exchange request still points at. The admin console attempts it and MySQL refuses.

> **WORTH DEMONSTRATING IN THIS ORDER**
>
> Delete a student in the admin console and watch the student count, the request count and the review count all fall together. Then try to delete a skill that is in use and watch the delete be refused. Those two clicks show cascade and restrict, which is the part of the schema that is hardest to explain on paper and easiest to show on screen.

13 The three views

A view is a stored query that behaves like a table. All three are used by the running site, not merely defined for the report.

VIEW	WHAT IT GIVES, AND WHO READS IT
`v_user_ratings`	Review count and average rating per student. This is why no average is stored on `users`. Read by the profile header, the rail, the search results and the admin table.
`v_request_details`	The request list with ids replaced by readable names and skill names. Read by the admin console's requests tab.
`v_session_overview`	Sessions with both partner names and the skill being taught.

14 Where each required SQL feature lives

`database/queries.sql` is grouped and numbered so a particular requirement can be found without reading the file end to end. Pasting it into the phpMyAdmin SQL tab runs every feature in turn; the write queries in group I are each wrapped in a transaction that rolls back, so the sample data is undisturbed afterwards.

FEATURE	GROUP
SELECT, WHERE, LIKE, ORDER BY, LIMIT	A1 – A3
INNER JOIN, up to five tables	B1 – B3
LEFT JOIN and RIGHT JOIN	C1 – C3
GROUP BY, HAVING, COUNT, SUM, AVG, MIN, MAX	D1 – D6
Scalar, IN, NOT EXISTS, correlated, derived-table and ALL subqueries	E1 – E6
The parameterised search queries the interface itself uses	F1 – F3
Views	G1 – G3
Indexes, with EXPLAIN proof that they are used	H1 – H3
INSERT, UPDATE, DELETE with cascade, and transactions	I1 – I5
Reporting queries behind the dashboard figures	J1 – J3

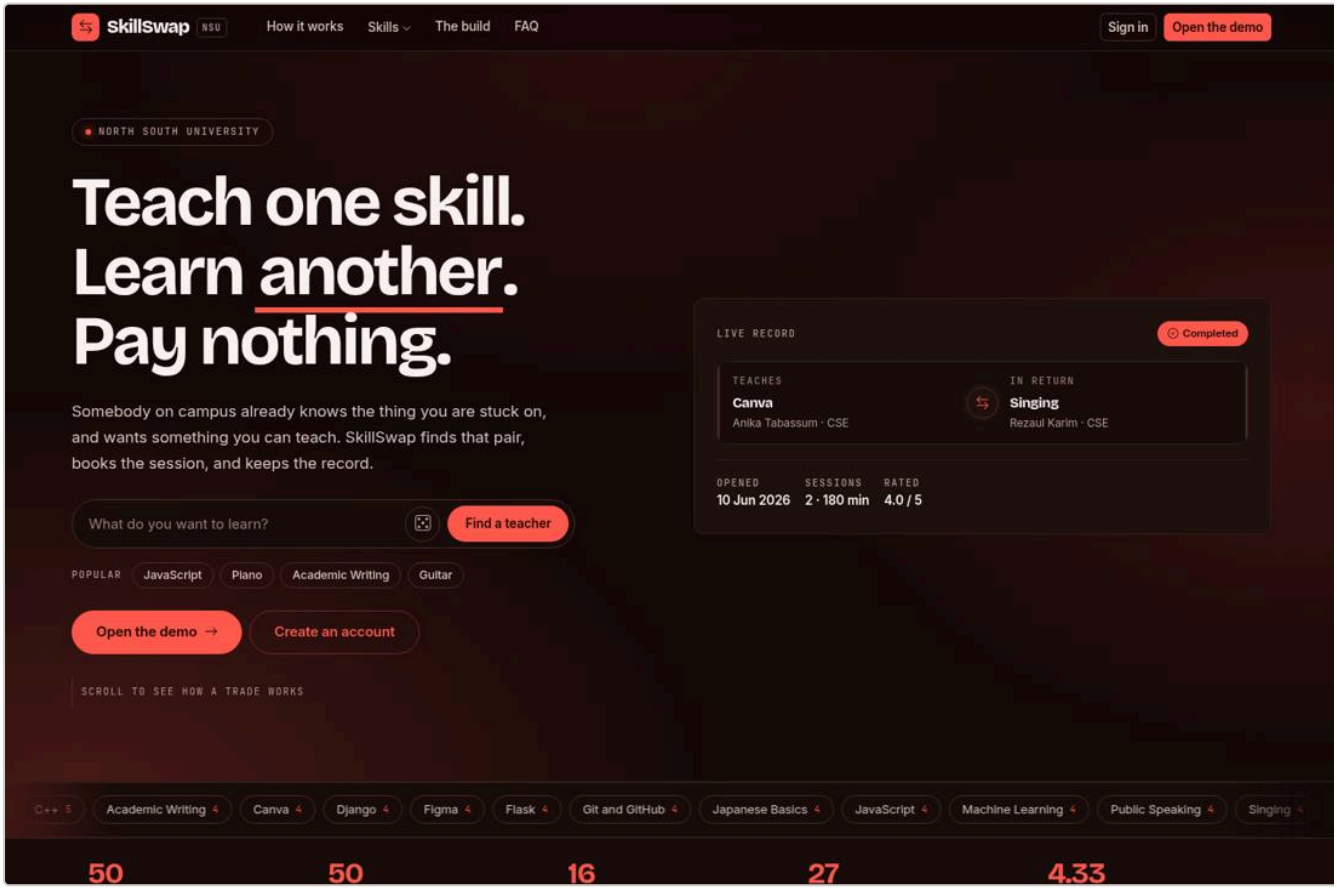
PART IV

What the website does

15 The eleven screens, one at a time

The landing page

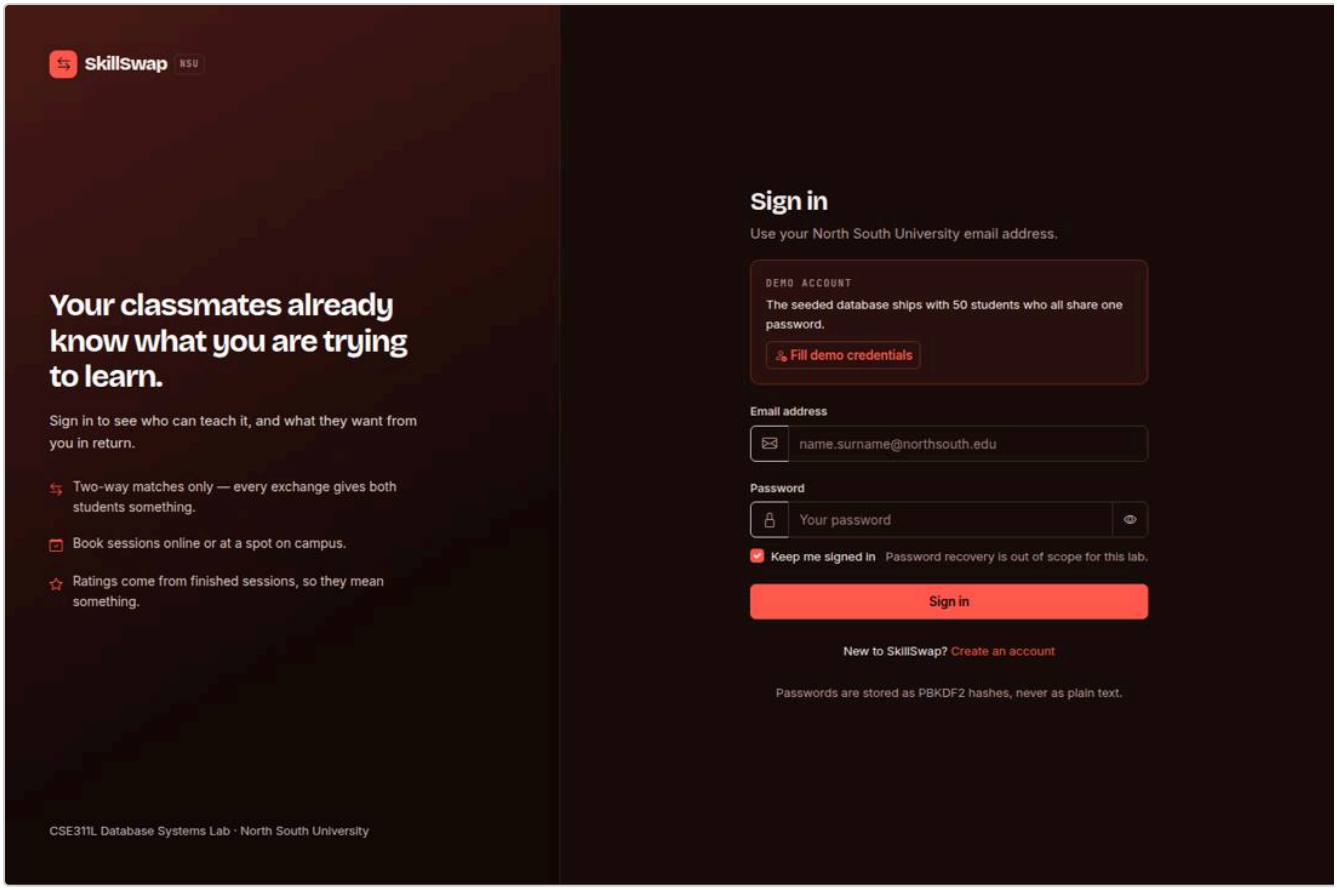
The public front page. Every figure on it is a query result rather than a fixed number: the counters come from `COUNT(*)` over each table, the moving band of skill names lists exactly those skills somebody has offered to teach, the category menu carries live per-category counts, and the sample record in the corner is a real completed exchange pulled out of the database with its session count and rating.



The landing page. The 50, 50 and 4.33 along the bottom are live counts.

Sign in and register

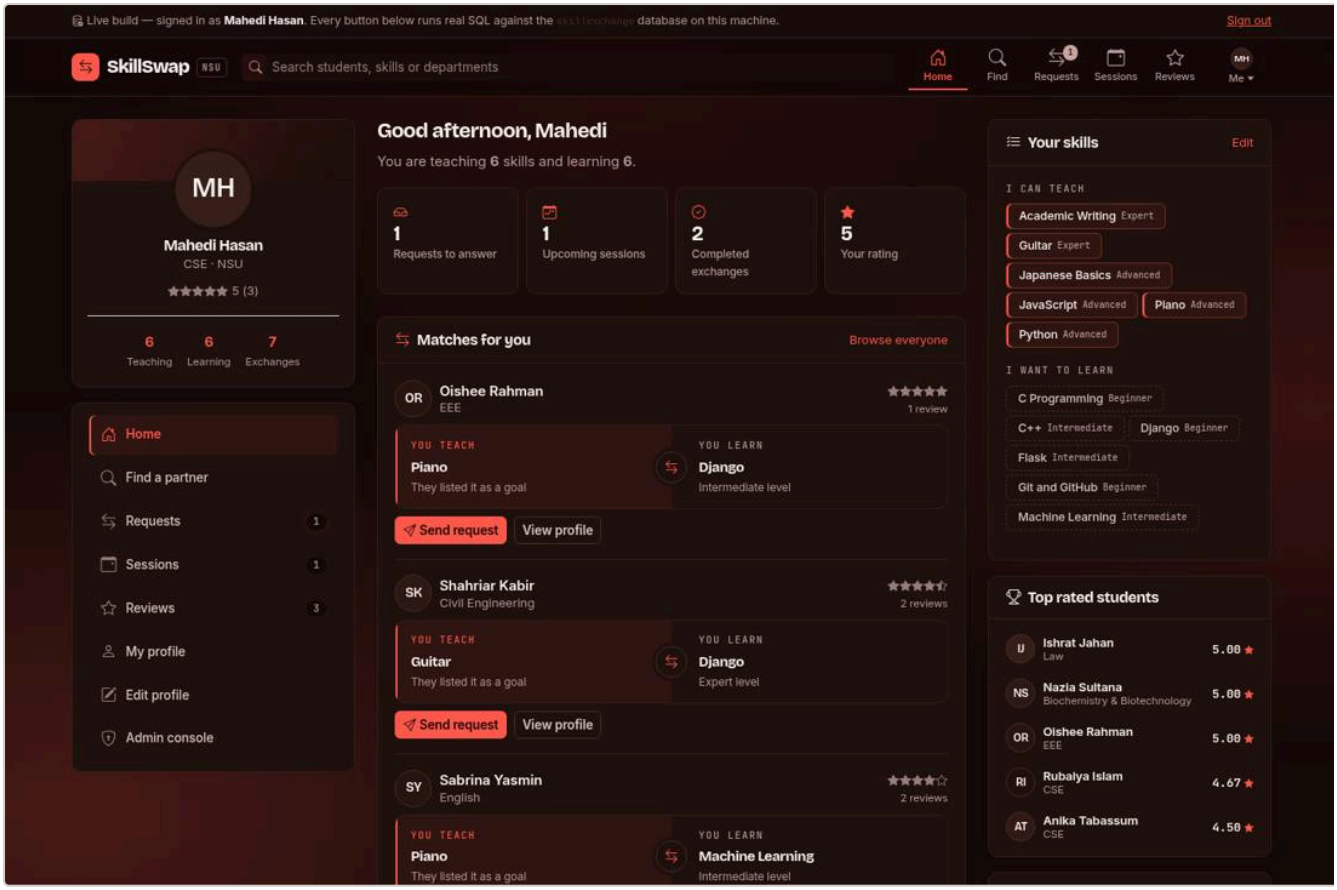
Sign in takes an NSU email address and a password, looks the row up by email, and verifies the submitted password against the stored PBKDF2 hash. All fifty seeded students share `password123`, and the **Fill demo credentials** button enters a working pair. Registration writes a real row: the new account appears in search and in the admin list immediately, and its password is hashed before it is stored.



Sign in. The panel at the top explains the shared demo password.

Dashboard

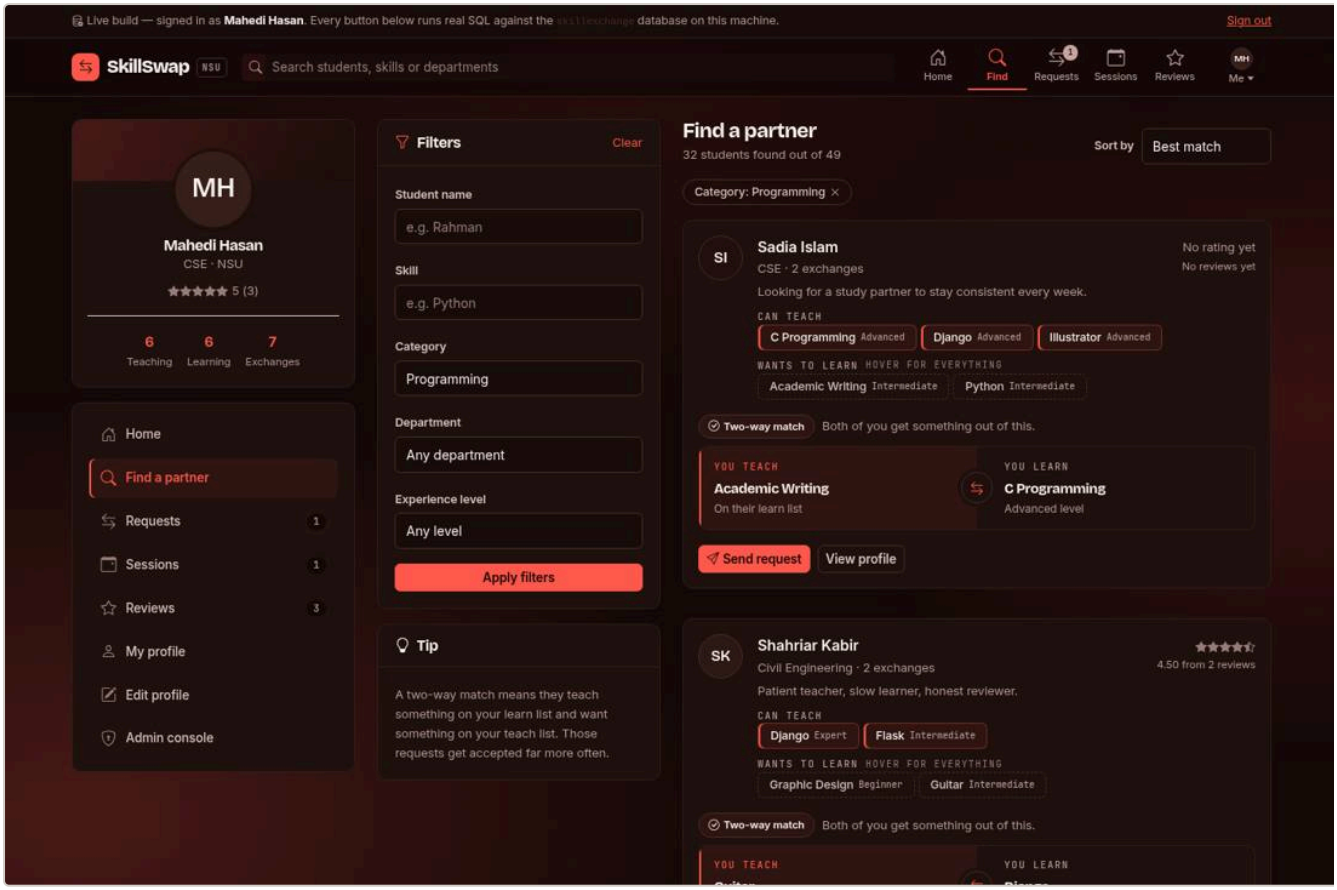
The signed-in home. The four tiles count requests waiting for an answer, upcoming sessions, completed exchanges and the student's own rating. Below them is the part that matters: **matches**. A match is somebody who teaches a skill this student wants to learn *and* wants to learn a skill this student can teach — a genuine two-way trade, found by MySQL rather than by a list of suggestions. Each match card shows both halves of the trade and offers to send the request straight away.



The dashboard. Each card is one two-way match, with both sides of the trade named.

Find a partner

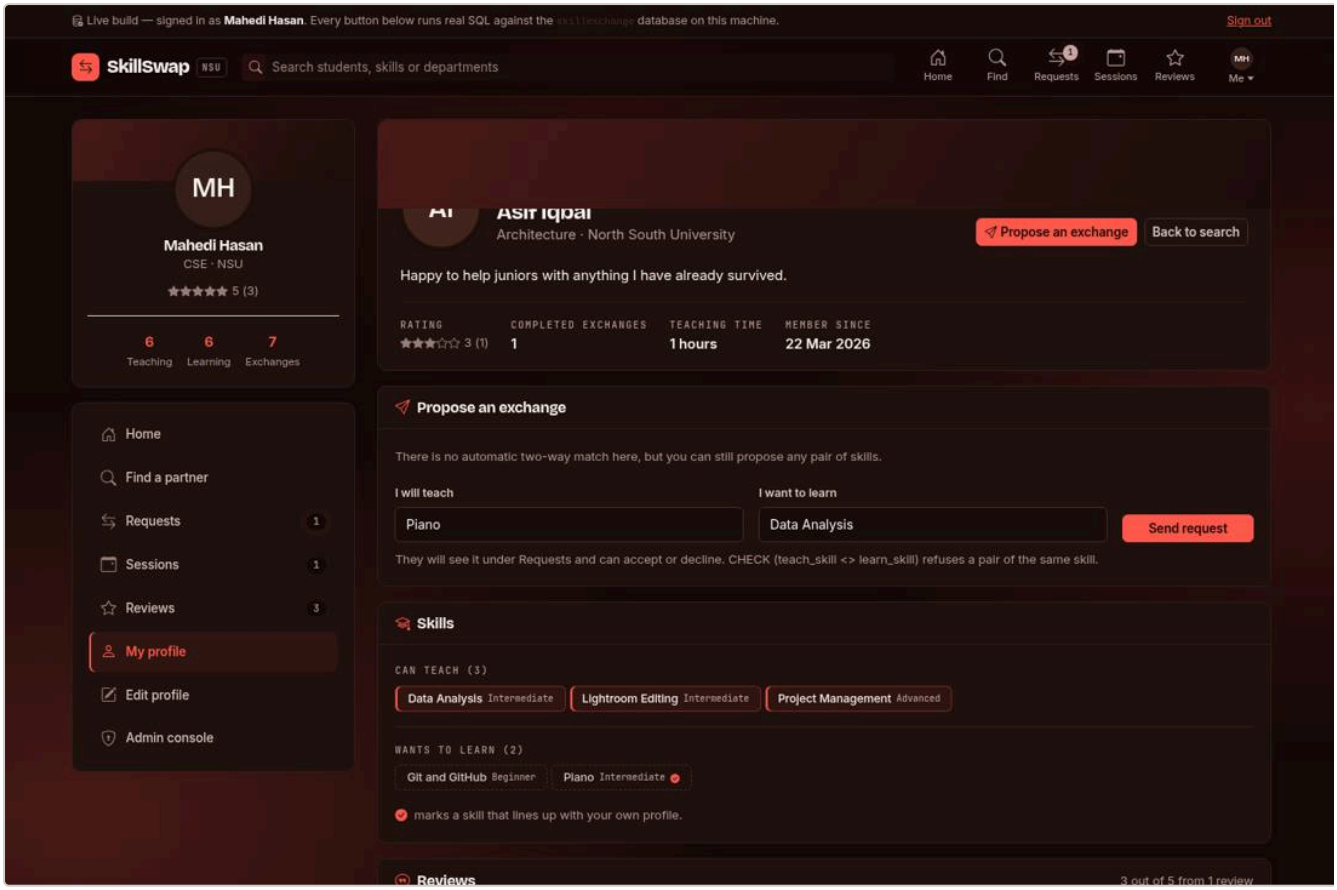
Search by free text, by a specific skill, by category, by department, or by proficiency level, in any combination. Every filter becomes a clause in one SELECT, so the browser is not filtering anything — changing a filter reloads the page with a different query.



Find a partner. Active filters appear as chips that remove themselves when clicked.

Profile

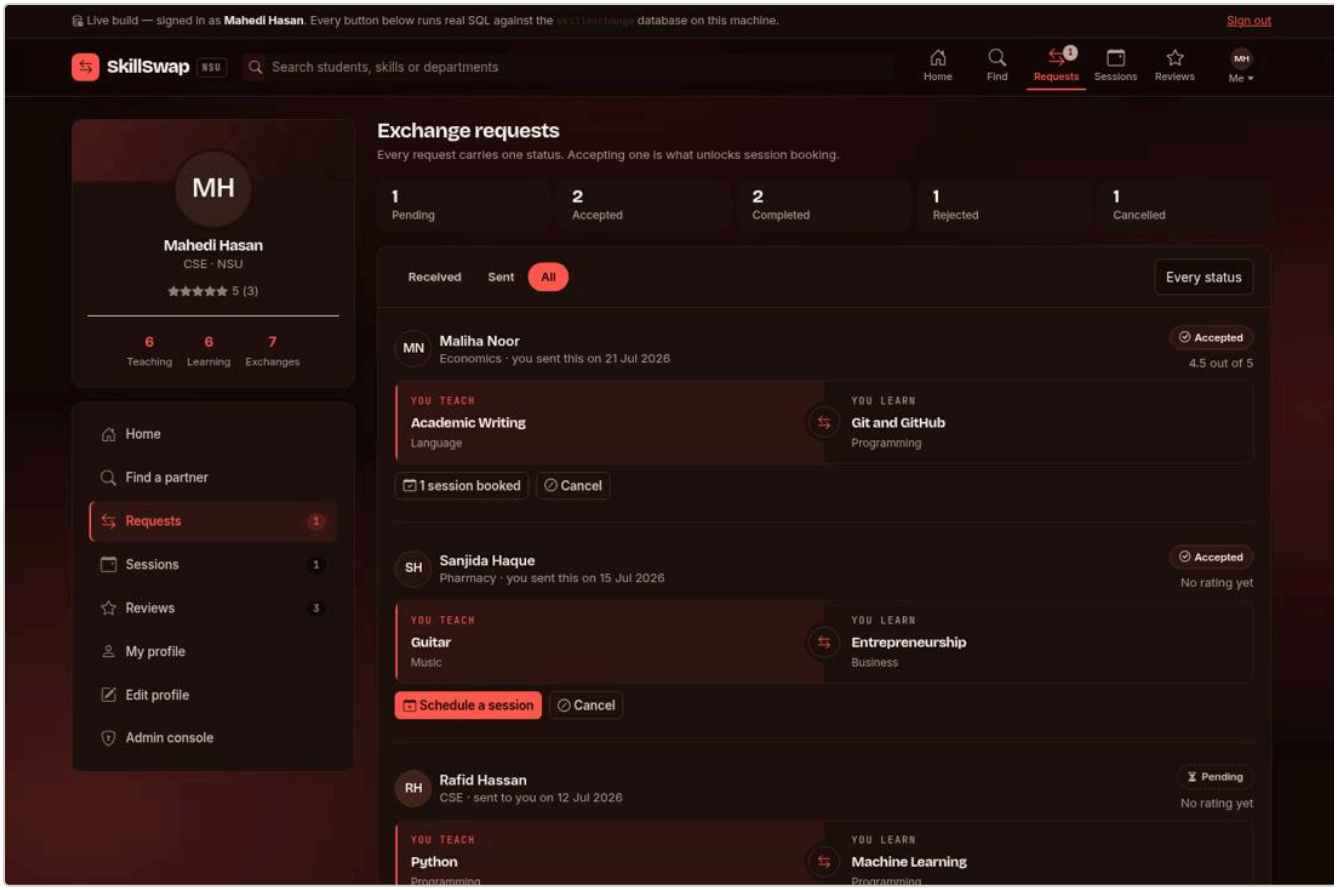
A student's public page, addressed as `profile.php?id=19`. It carries their skills in both directions, the reviews they have received with the rating spread, their exchange history, and a form to propose a trade. When a request between the two students is already pending or accepted, the form is replaced by a note saying so, because the database would refuse a duplicate anyway.



A public profile, seen by another student.

Requests

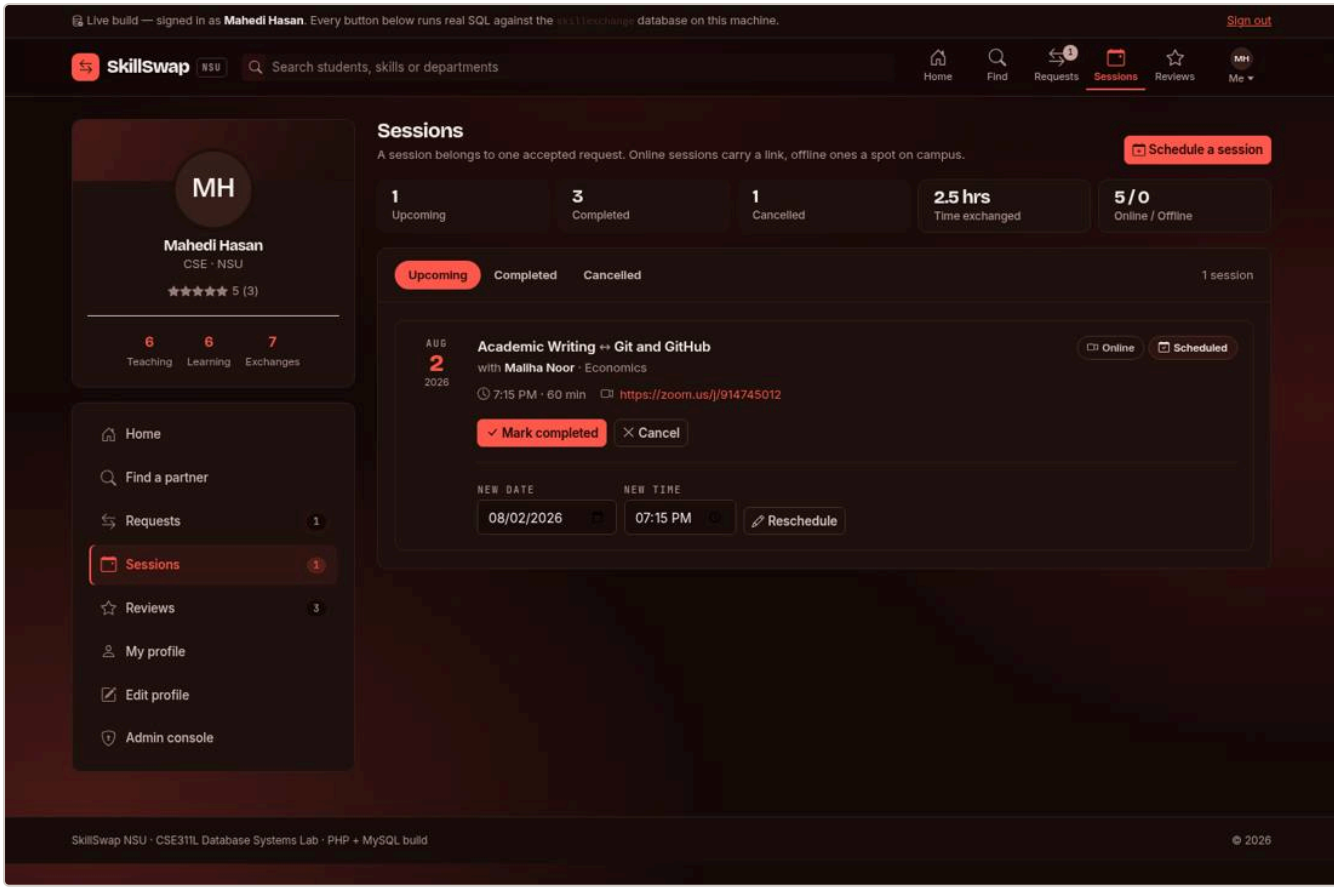
Every exchange request this student is part of, split into received, sent and all, filterable by status. A request carries exactly one status at a time, and the buttons available depend on it: a pending request that arrived can be accepted or declined, a pending request that was sent can only be cancelled, and an accepted request offers to schedule a session. Accepting is what unlocks booking — a session cannot exist without an accepted request behind it.



Exchange requests. The tiles across the top are per-status counts and double as filters.

Sessions

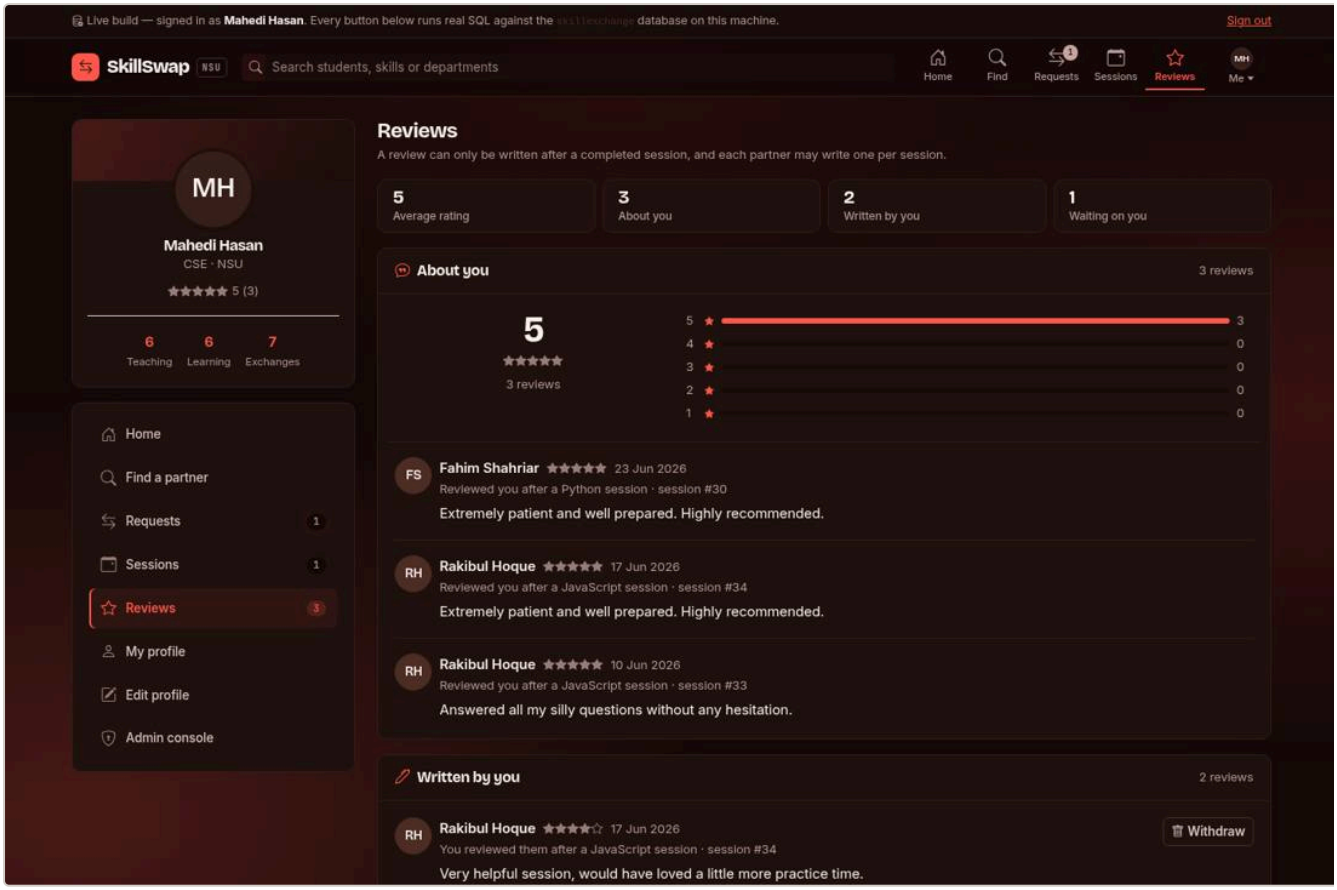
The booked meetings, split into upcoming, completed and cancelled. Booking asks for a date, a time, a duration and whether the session is online or in person, and the field beneath changes between a meeting link and a location accordingly. A scheduled session can be marked completed, cancelled, or moved to another slot. Completing the last open session of an exchange also completes the exchange, inside a transaction.



Sessions. The counters include total time exchanged and the online/offline split.

Reviews

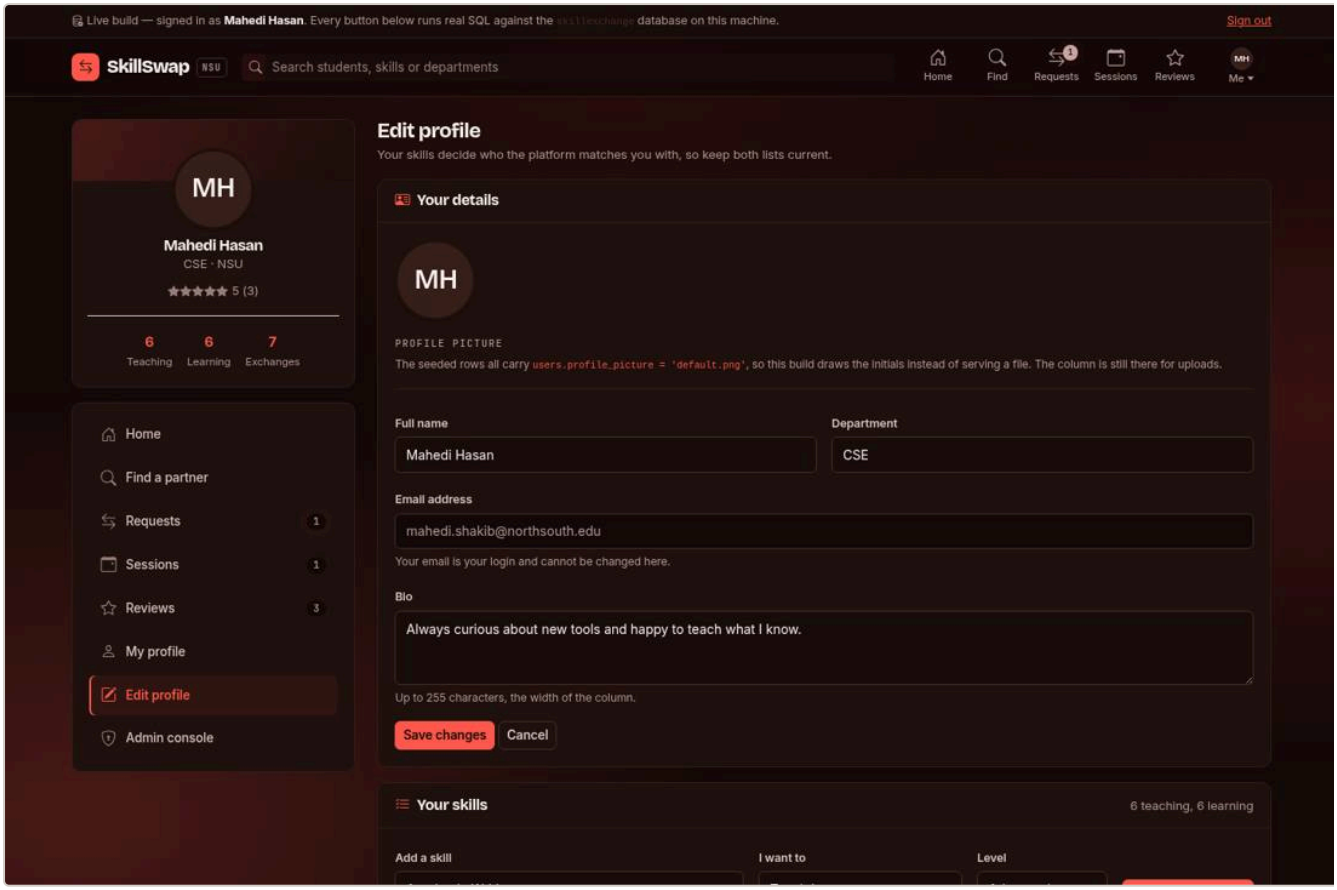
Reviews received, reviews written, and the sessions still waiting for one. That last list is a `NOT EXISTS` query: completed sessions for which this student has not yet written a review. A review belongs to a session, so it cannot be written before the session is completed, and the database allows exactly one per person per session.



Reviews, with the rating spread and the sessions still awaiting one.

Edit profile

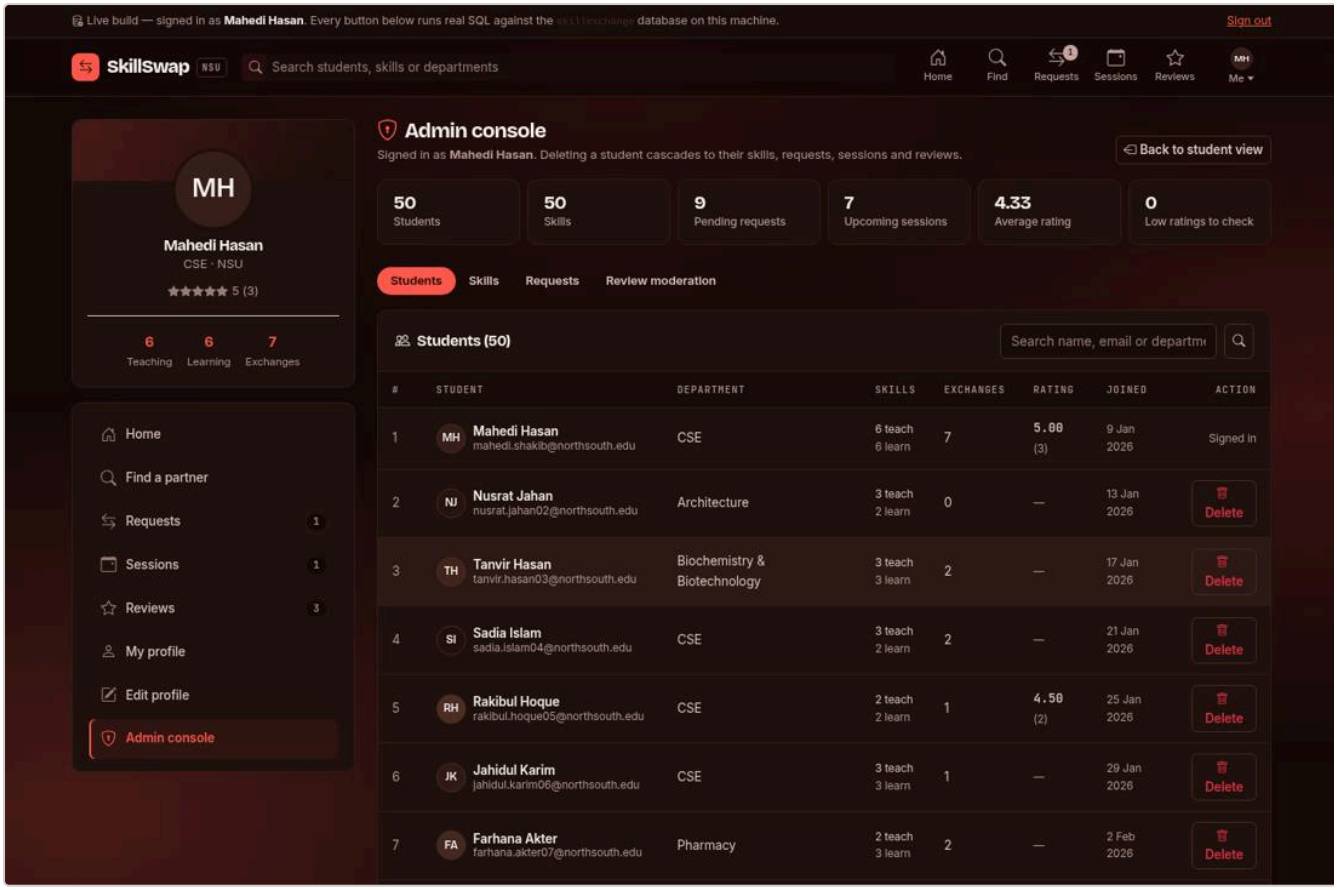
Three panels: the student's own details, their skill lists, and a password change. Skills can be added from the catalogue, have their proficiency changed, or be removed — each one a single statement against `userskills`. The password form checks the current password against the stored hash before writing a new hash.



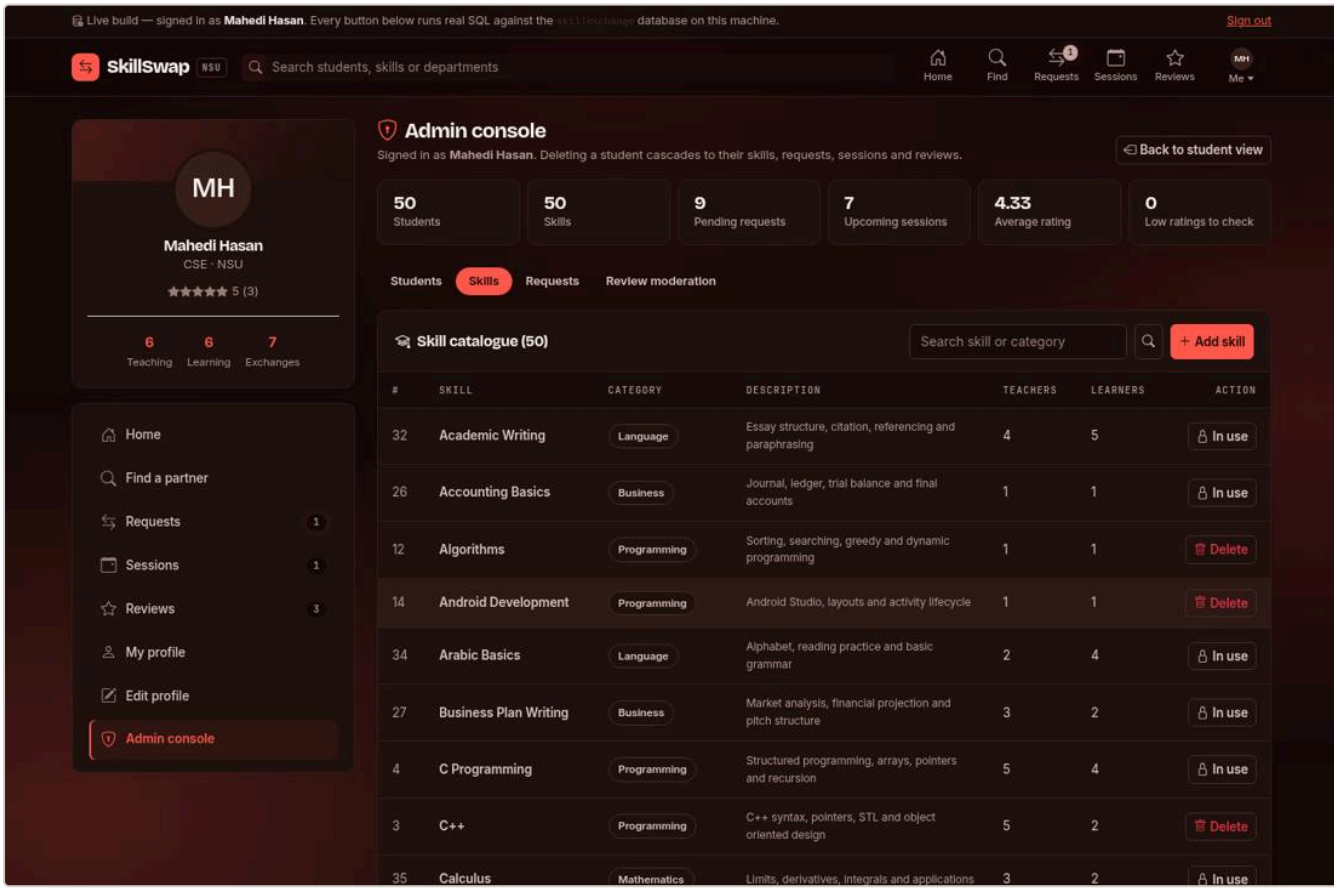
Edit profile. Every row in the skills table is one `userskills` row.

Admin console

Four tabs over the whole database: students, the skill catalogue, every exchange request, and review moderation. This is the screen where the schema is easiest to demonstrate, because deleting a student here runs one statement and the cascade does the rest, while deleting a skill that is still referenced is refused outright.



The admin console. Fifty students, each row with its skill, exchange and rating counts.



The skills tab. Skills an exchange request still points at cannot be deleted.

PART V

One exchange, end to end

16 Request #22, from proposal to rating

The clearest way to show that the whole stack is connected is to follow one exchange through every table it touches. This one is in the seeded data, so it can be followed on any machine after a fresh import.

The two students. Mahedi Hasan (`user_id` 1) teaches Python at Advanced level and wants to learn Machine Learning at Intermediate level. Rafid Hassan teaches Machine Learning and wants to learn Python. Each has exactly what the other is missing, which is what makes this a two-way match rather than a favour.

Step 1 — The match is found

Before any request exists, the dashboard finds this pair by asking MySQL for students who teach something in my Learn list and want something in my Teach list. Both halves have to hold, which is why the query has two `IN` subqueries against `userskills`.

```
-- simplified: the shape of matches_for()
SELECT u.user_id, u.name, theirs.skill_name AS they_teach, mine.skill_name AS i_teach
FROM   users u
       JOIN (teach rows) theirs ON theirs.user_id = u.user_id
       JOIN (my teach rows) mine
            ON mine.skill_id IN (SELECT skill_id FROM userskills
                                 WHERE user_id = u.user_id AND skill_type = 'Learn')
WHERE  theirs.skill_id IN (SELECT skill_id FROM userskills
                           WHERE user_id = ? AND skill_type = 'Learn');
```

Step 2 — The request is sent

Rafid presses **Send request**. One row appears in `exchangerequests` with status `Pending`. The `teach_skill` column always belongs to the sender, which is why the two students see the same row labelled in opposite directions.

```
INSERT INTO exchangerequests (sender_id, receiver_id, teach_skill, learn_skill, status)
VALUES (?, ?, ?, ?, 'Pending');
```

At this moment Mahedi's navigation bar grows a small counter, because the top bar runs `SELECT COUNT(*) FROM exchangerequests WHERE receiver_id = ? AND status = 'Pending'` on every page load.

Step 3 — The request is accepted

Mahedi opens the requests screen and presses **Accept**. One statement runs, and the card immediately offers to schedule a session instead — because the buttons a request shows are decided by its status, and its status just changed.

```
UPDATE exchangerequests SET status = 'Accepted' WHERE request_id = 22;
```

Step 4 — A session is booked

Either student books a slot: a date, a time, a duration in minutes, and online or in person. The row belongs to the request, which is how the session knows who is meeting whom and about what.

```
INSERT INTO sessions (request_id, session_date, session_time, duration, mode, meeting_link)
VALUES (?, ?, ?, ?, 'Online', ?);
```

Booking the identical slot a second time is refused here by `UNIQUE (request_id, session_date, session_time)`, and a duration of nine hours is refused by `CHECK (duration BETWEEN 15 AND 480)`. Neither refusal comes from PHP.

Step 5 — The session is completed

Afterwards the session is marked completed. Because it was the only open session on this exchange, the exchange is completed too, and the two updates run inside one transaction so that a failure cannot leave a finished session attached to an unfinished exchange.

```
START TRANSACTION;
UPDATE sessions         SET status = 'Completed' WHERE session_id = ?;
UPDATE exchangerequests SET status = 'Completed' WHERE request_id = ?;
COMMIT;
```

Step 6 — They rate each other

The completed session now appears on both students' reviews screens under “waiting for your review” — that list is the `NOT EXISTS` query. Each writes one review.

```
INSERT INTO reviews (session_id, reviewer_id, reviewee_id, rating, comment)
VALUES (?, ?, ?, ?, ?);
```

A second review from the same person is refused by `UNIQUE (session_id, reviewer_id)`, and reviewing yourself is refused by `CHECK (reviewer_id <> reviewee_id)`.

Step 7 — The rating appears everywhere at once

Nothing is copied anywhere. Both students' averages change immediately on their profiles, in the search results, in the left rail and in the admin table, because none of those places stores an average — they all read `v_user_ratings`, which recomputes it from the `reviews` table on every read. That is the practical payoff of third normal form, and it is visible on screen the moment the review is published.

STEP	TABLE TOUCHED	STATEMENT
Send request	`exchangerequests`	INSERT
Accept	`exchangerequests`	UPDATE
Book	`sessions`	INSERT
Complete	`sessions` + `exchangerequests`	Two UPDATEs in a transaction
Review	`reviews`	INSERT
Rating shown	`v_user_ratings`	SELECT — nothing is stored

PART VI

Demonstrating the project

17 A suggested order, and what to point at

Fifteen minutes is enough to show every idea in the project if the order is right. Each step below produces a visible change, and in every case the message in the corner names the statement that produced it.

Open with the data, not the design

Show phpMyAdmin first, with the six tables and their row counts, then open the admin console in the browser and show the same numbers. That establishes immediately that the website is reading this database and not a copy.

Then the read path

Open the dashboard and explain one match card: this student teaches something I want, and wants something I teach, and MySQL found that pair. Then use the search filters — pick a category and a proficiency level — and point out that each change is a new query, not JavaScript hiding rows. Turning JavaScript off in the browser and reloading makes the point conclusively.

Then the write path

Accept a pending request and read the message in the corner aloud: it is the `UPDATE` that just ran. Then book a session against that accepted request, and immediately try to book the same date and time again — the refusal comes from `UNIQUE (request_id, session_date, session_time)`. Mark the session completed and point out that the request became completed too, in the same transaction. Then leave a review, and try to leave a second one on the same session.

Then the constraints

In the admin console, delete a student and watch the student, request and review counts all fall together — that is one `DELETE FROM users` and `ON DELETE CASCADE`. Then try to delete a skill that an exchange request still points at, and show the refusal from `ON DELETE RESTRICT`.

Close with security

Open `includes/queries.php` and show that every value is a `?`. Then type `' OR 1=1 --` into the search box and show that it simply finds nothing, because a bound value can never become syntax. Finally show the `password` column in phpMyAdmin: PBKDF2 hashes, no plain text anywhere.

18 Restoring the sample data afterwards

Deleting a student during a demonstration really deletes them. To get the sample data back, import `database/skillexchange_full.sql` again — it drops and recreates everything, so the row counts return to 50, 50, 261, 49, 34 and 36. Doing that once before the demonstration begins is a good habit, so that the numbers on screen match the numbers in the report.

> **THE TWO BUILDS, AND WHY THERE ARE TWO**
>
> A static version of the same eleven screens is published at `mahedi-123.github.io/SkillSwapNSU/` so that the interface has a public link. GitHub Pages serves files and runs no code, so that version reads a JavaScript export of `seed.sql` and keeps its changes inside the browser tab. It is the interface without a database behind it. The folder described in this document is the connected build: the same screens, backed by MySQL, where the writes actually happen and persist.
